Networking Fundamentals

ICMP • IP Fragmentation • IPv4 Addressing • Subnetting • IPv6 • Routing

A beginner-to-intermediate guide with diagrams, hands-on commands and real-world CVEs and incidents

What you will learn

- How **ICMP** powers ping, traceroute and redirects, and how attackers abuse it
- Why packets get **fragmented** and reassembled, and how that has caused real vulnerabilities
- IPv4 structure, **address classes**, **subnet masks** and **private addresses** with worked examples
- **IPv6** address types, shortening rules and how it differs from IPv4
- A hands-on **routing lab** you can build on any Linux machine, plus the four **modes of addressing**
- A command cheat sheet, practice questions with answers, and a table of recent industry references

> **Lab safety note**
>
> - Run capture and test commands only on your own lab or on networks you are authorised to test.
> - Examples use documentation and private ranges (192.168.x.x, 10.x.x.x, 203.0.113.x, 198.51.100.x, 2001:db8::/32). Replace them with your lab values.

Contents

1. Understanding ICMP Messages 3

2. IP Fragmentation and Reassembly 6

3. IPv4 7

4. Classes of IP Addresses 8

5. Subnet Masks 9

6. Private IP Addresses 11

7. Understanding IPv6 Addresses 12

8. Practical: Routing and Modes of Addressing 13

9. IPv4 vs. IPv6 15

10. Industry Reference Summary 15

11. Command Cheat Sheet 16

12. Practice Questions 17

13. References and Further Reading 18

1. Understanding ICMP Messages

ICMP (Internet Control Message Protocol) is IP's built-in error-reporting and diagnostic helper. It does not carry user data. Instead, routers and hosts use it to say things like “that host is unreachable”, “your packet expired” or “I am alive”. ICMP rides directly inside an IP packet (IP protocol number 1). IPv6 uses ICMPv6 (next-header 58), and it is far more important there.

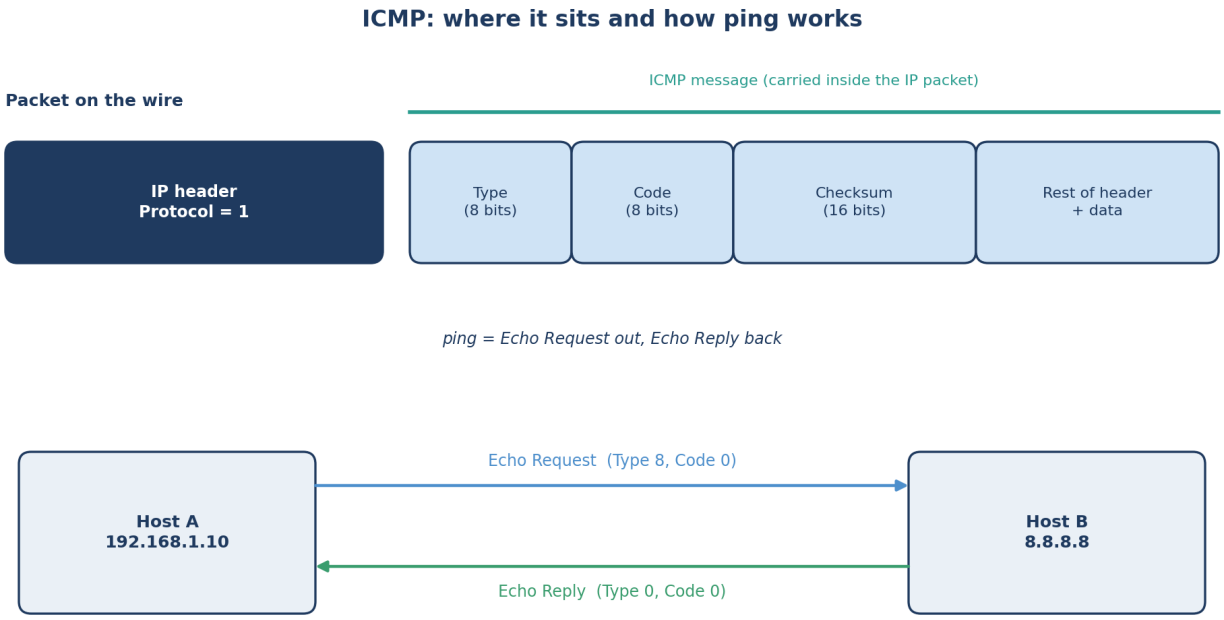


Figure 1. An ICMP message sits inside an IP packet. Ping is an Echo Request answered by an Echo Reply.

Common ICMP message types

Type	Name	What it means / where you see it
0	Echo Reply	Answer to a ping
3	Destination Unreachable	Code 0 network, 1 host, 3 port unreachable, 4 **fragmentation needed but DF set** (used by Path MTU Discovery)
5	Redirect	A router suggests a better next hop on the same LAN
8	Echo Request	A ping
11	Time Exceeded	TTL reached 0 in transit. This is how traceroute discovers routers
ICMPv6	128/129 echo, 133/134 router solicit/advert, 135/136 neighbor solicit/advert, 2 packet too big	ICMPv6 also does address resolution (replaces ARP), autoconfiguration (SLAAC) and path MTU, so it cannot simply be blocked

1.1 ICMP Ping

Ping measures reachability and round-trip time (RTT). Read the output like this: **ttl** hints at the remote OS (about 64 for Linux, 128 for Windows, minus hops travelled), **time** is RTT in milliseconds, and the summary line shows packet loss.

```
# Linux / macOS
ping -c 4 8.8.8.8                # send 4 echo requests
ping -c 4 -s 1000 8.8.8.8        # custom payload size
ping -M do -s 1472 8.8.8.8       # set DF bit: 1472 + 28 header bytes = 1500 (tests path MTU)

# Windows
ping -n 4 8.8.8.8
ping -f -l 1472 8.8.8.8          # -f = don't fragment, -l = payload size

# IPv6
ping -6 2001:4860:4860::8888     # Linux also has ping6
```

1.2 Traceroute

Traceroute sends probes with TTL = 1, 2, 3 and so on. Every router decrements the TTL. The router that drops it to 0 sends back **ICMP Time Exceeded (Type 11)**, revealing its address. When a probe finally reaches the destination, it answers with an Echo Reply (ICMP mode) or Port Unreachable (UDP mode).

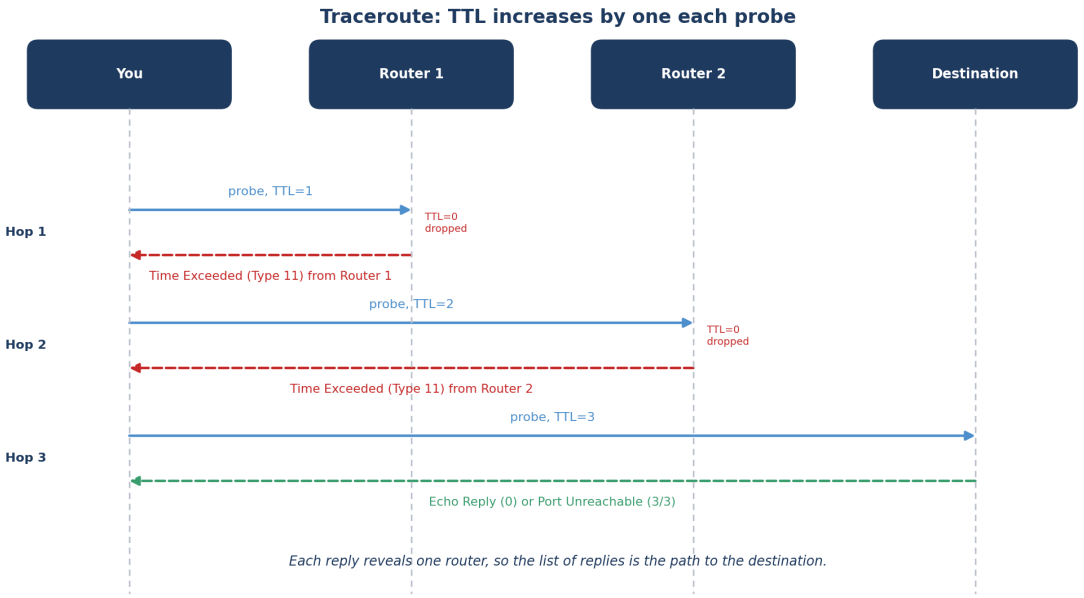


Figure 2. Each probe travels one hop further than the last, so each Time Exceeded reply names the next router.

```
traceroute -n 8.8.8.8          # Linux default: UDP probes to high ports (33434 and up)
traceroute -I -n 8.8.8.8       # use ICMP Echo probes
traceroute -T -p 443 host      # TCP SYN to port 443 (passes more firewalls)
tracert -d 8.8.8.8             # Windows (uses ICMP Echo, -d = no DNS lookups)
mtr -n 8.8.8.8                 # live traceroute with per-hop loss statistics
```

- * * * means no reply for that hop. The router may filter or rate-limit ICMP, which does not necessarily mean the path is broken.
- Different probe types can show different paths, because load balancers hash on ports and protocol.

1.3 ICMP Redirect

If a host sends a packet to its default gateway, and the gateway knows a better router on the *same subnet*, it forwards the packet and also sends an ICMP Redirect (Type 5) telling the host to use the better gateway next time.

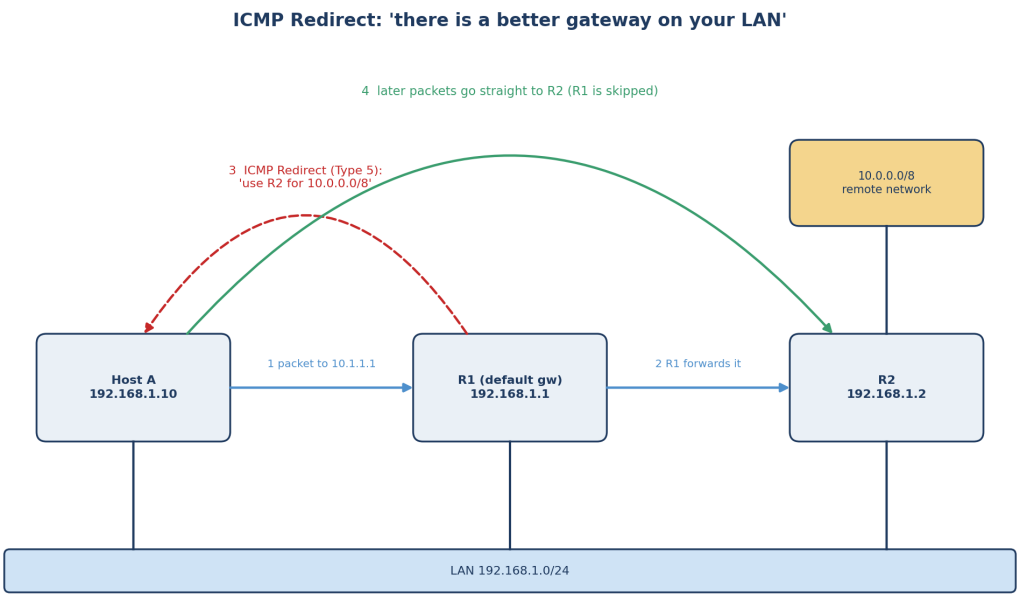


Figure 3. R1 forwards the first packet, then tells Host A to use R2 directly for that destination.

Security concern: an attacker on the same LAN can forge redirects to steer your traffic through their machine (a man-in-the-middle). Most hardened systems ignore redirects.

```
# Linux: ignore and do not send redirects
sudo sysctl -w net.ipv4.conf.all.accept_redirects=0
sudo sysctl -w net.ipv4.conf.all.send_redirects=0
sudo sysctl -w net.ipv6.conf.all.accept_redirects=0

# Windows
netsh interface ipv4 set global icmpredirects=disabled

# Detect ICMP anomalies on Linux (large ICMP packets are suspicious)
sudo tcpdump -ni eth0 'icmp and greater 100'
```

Real-world references: ICMP

Defender takeaways: baseline normal ICMP sizes and rates, alert on large or frequent echo payloads, block outbound ICMP where not needed, and keep OS patches current because ICMP is parsed inside the kernel.

- **PingPull (Gallium APT):** a backdoor that uses ICMP Echo Request and Echo Reply packets to receive commands and return results. Unit 42 noted that few organisations inspect ICMP, which makes this channel hard to spot.
- **Pingback:** Windows malware that listens for inbound ICMP packets and parses commands using sequence numbers such as 1234 to 1236 (analysed by Trustwave).
- **BPFDoor variants:** newer Linux backdoor variants reportedly use an ICMP relay, relying on the fact that networks often allow ping and rarely inspect it deeply (Rapid7 analysis).
- **CVE-2020-16898 “Bad Neighbor” (CVSS 8.8):** Windows mishandled ICMPv6 Router Advertisements that carried the RDNSS option (type 25) with an even length value, which enabled remote code execution. Patched on 13 October 2020.

2. IP Fragmentation and Reassembly

Every link has a **Maximum Transmission Unit (MTU)**, usually 1500 bytes on Ethernet. If an IPv4 packet is larger than the next link's MTU, a router may split it into **fragments**. Only the **destination host** puts them back together. Three header fields make this work (see Figure 4 and the header in Section 3):

- **Identification:** the same value on every fragment of one original datagram
- **Flags:** DF (Don't Fragment) and MF (More Fragments; set on all but the last fragment)
- **Fragment Offset:** where this piece belongs, counted in units of 8 bytes

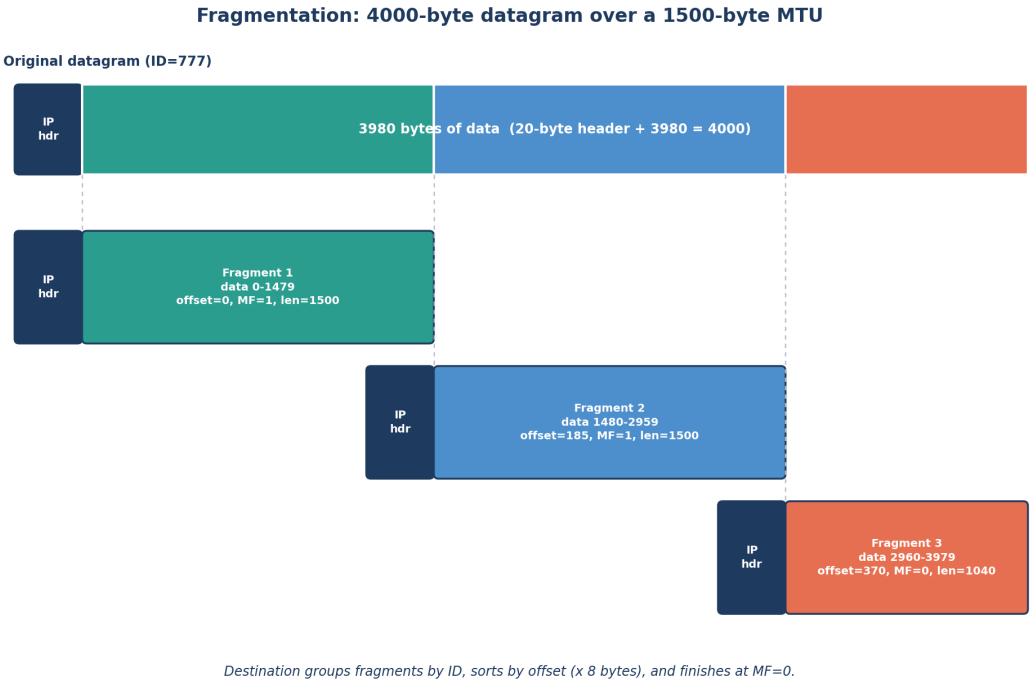


Figure 4. Worked example: 3980 data bytes cross a 1500-byte MTU. Offsets are 0, 185 and 370 (that is, 0, 1480 and 2960 bytes divided by 8).

```
# Generate fragments (3972 ICMP data + 8 ICMP header + 20 IP header = 4000 bytes)
ping -c 1 -s 3972 192.168.1.1

# Capture only fragments
sudo tcpdump -ni eth0 'ip[6:2] & 0x3fff != 0'
# Wireshark display filter:
#   ip.flags.mf == 1 || ip.frag_offset > 0

# Discover the path MTU
ping -M do -s 1472 host        # lower -s until replies succeed
tracepath host                 # shows PMTU per hop
```

Path MTU Discovery (PMTUD)

With DF set, a router that cannot forward the packet drops it and returns ICMP Type 3 Code 4 (“fragmentation needed”). The sender then lowers its packet size. If a firewall blocks all ICMP, this message never arrives and large transfers hang. This is the classic symptom “pings work, but big web pages or VPN traffic stall”. **IPv6 routers never fragment.** Only the sender may, and ICMPv6 “Packet Too Big” (Type 2) is essential.

Real-world references: fragmentation

Defender takeaways: patch the TCP/IP stack, rate-limit fragments at the edge, and make sure security devices reassemble traffic before inspection.

- **CVE-2018-5391 (“FragmentSmack”):** a stream of tiny IP fragments could exhaust the reassembly queue on Linux and Windows and degrade or deny service. One Linux mitigation was to reduce the fragment queue thresholds with `net.ipv4.ipfrag_low_thresh` and `ipfrag_high_thresh`.
- **CVE-2021-24074, CVE-2021-24086, CVE-2021-24094 (Windows tcpip.sys):** flaws in IPv4 and IPv6 reassembly. 24074 was a potential remote code execution caused by fragments carrying different IP options; the other two led to denial of service.
- **jolt2 (CVE-2000-0305)** and the older Ping of Death and Teardrop attacks show that reassembly bugs keep returning across decades.
- **Evasion:** attackers split payloads into fragments so naive IDS or firewalls miss them. Good devices reassemble before inspecting.

3. IPv4

IPv4 uses a **32-bit address** written as four 8-bit octets in dotted decimal, for example **192.168.10.25**. That gives about 4.3 billion addresses (2^{32}), which is why the world is moving to IPv6. Each address has a **network part** (identifies the network) and a **host part** (identifies the device); the subnet mask tells you where the split is.

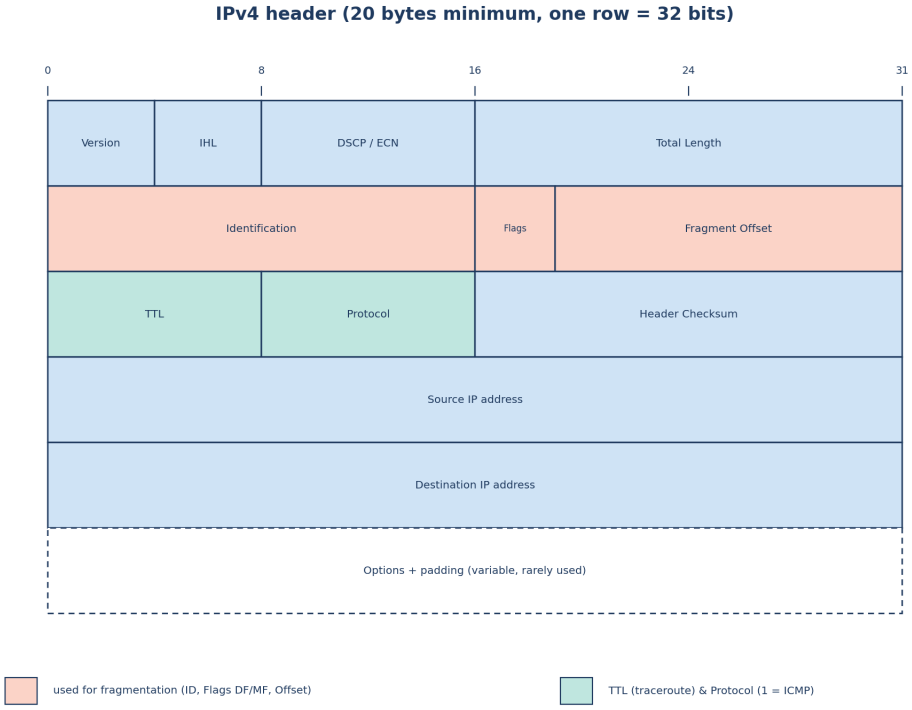


Figure 5. The IPv4 header. Orange fields drive fragmentation; green fields are used by traceroute (TTL) and identify ICMP (Protocol = 1).

```
# Show your IPv4 configuration
ip -4 addr show              # Linux
ifconfig                     # older Linux / macOS
ipconfig /all                # Windows

# Convert an address to binary quickly (Python)
python3 -c "print('.'.join(f'{int(o):08b}' for o in '192.168.10.25'.split('.')))"
```

- **TTL** (Time To Live) limits packet lifetime and prevents routing loops.
- **Protocol** tells the receiver what is inside: 1 = ICMP, 6 = TCP, 17 = UDP.
- The header is at least **20 bytes**; options are rare today.

4. Classes of IP Addresses

Originally IPv4 was divided into **classes** using the first bits of the first octet. Each class had a fixed network/host split and a **default mask**.

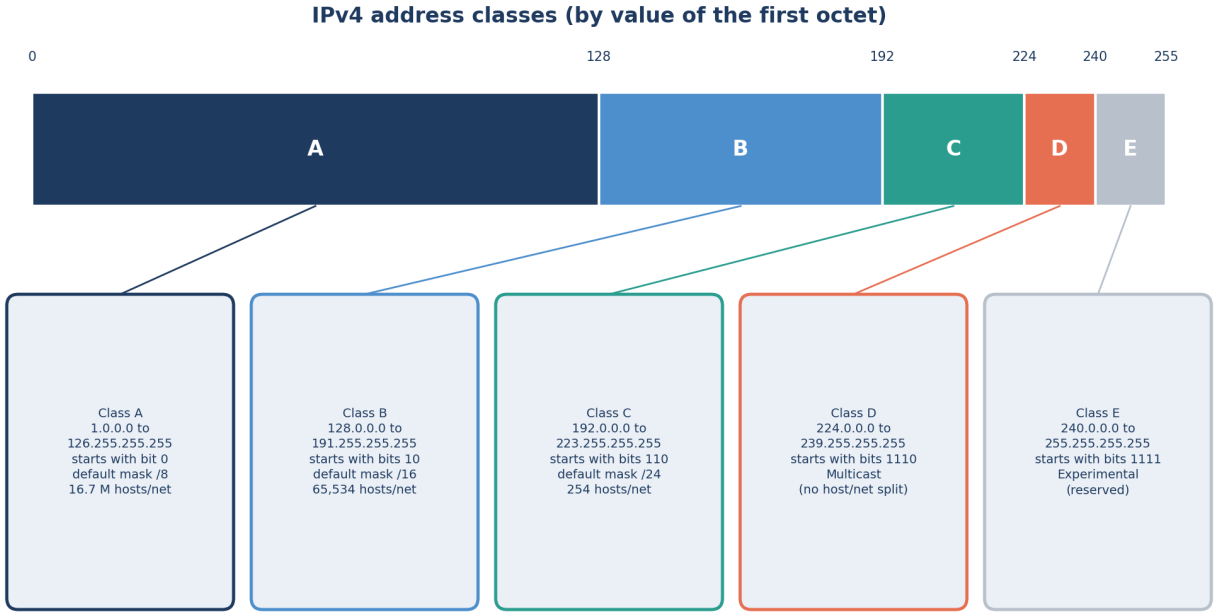


Figure 6. The first octet (0 to 255) decides the class. Class A takes half of all addresses, B a quarter, C an eighth.

Class	First octet	Leading bits	Default mask	Hosts per network	Typical use
A	1 to 126	0	255.0.0.0 (/8)	about 16.7 million	Very large organisations
B	128 to 191	10	255.255.0.0 (/16)	65,534	Medium organisations
C	192 to 223	110	255.255.255.0 (/24)	254	Small networks
D	224 to 239	1110	not applicable	not applicable	Multicast groups
E	240 to 255	1111	not applicable	not applicable	Reserved / experimental

- **127.0.0.0/8** is loopback (127.0.0.1 is “this machine”), so Class A effectively ends at 126.
- Classful addressing wasted many addresses, so it was replaced in 1993 by **CIDR** (Classless Inter-Domain Routing). Today you see /22, /27 and so on regardless of class.
- Classes are still taught because they explain default masks and appear in many exams.

5. Subnet Masks

A **subnet mask** is a 32-bit pattern of 1s followed by 0s. The 1s mark the network portion; the 0s mark the host portion. A device ANDs its address with the mask to find its network ID. The slash form (**/26**) just counts the 1s.

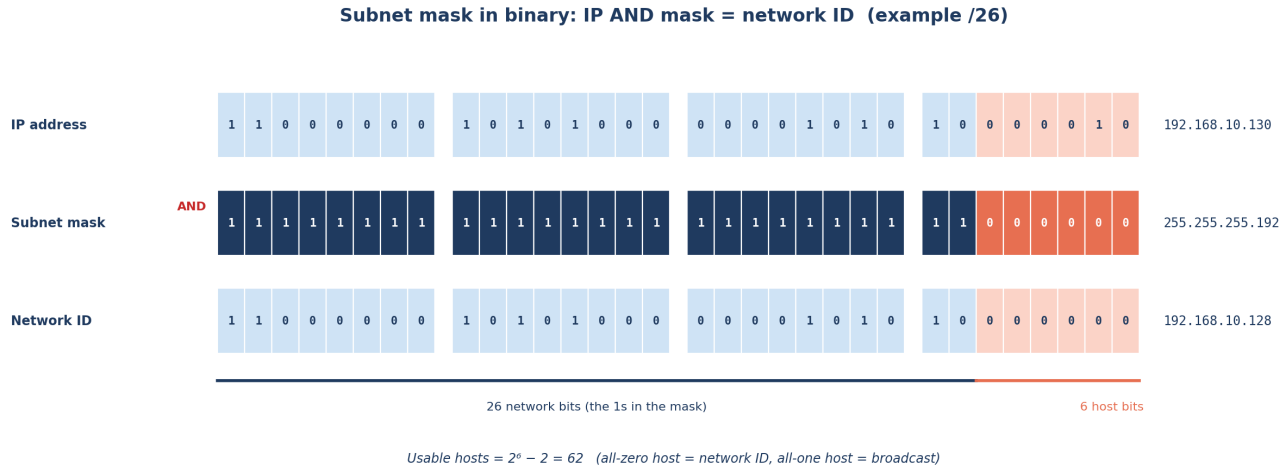


Figure 7. 192.168.10.130 with mask 255.255.255.192 (/26) belongs to network 192.168.10.128.

Two formulas to remember

- Usable hosts per subnet = $2^h - 2$, where h = number of host bits (minus the network ID and the broadcast address).
- Number of subnets = 2^b, where b = number of bits borrowed from the host part.
- Shortcut: **block size = 256 − (mask octet value)**. For 255.255.255.192 the block size is 64, so subnets start at .0, .64, .128, .192.

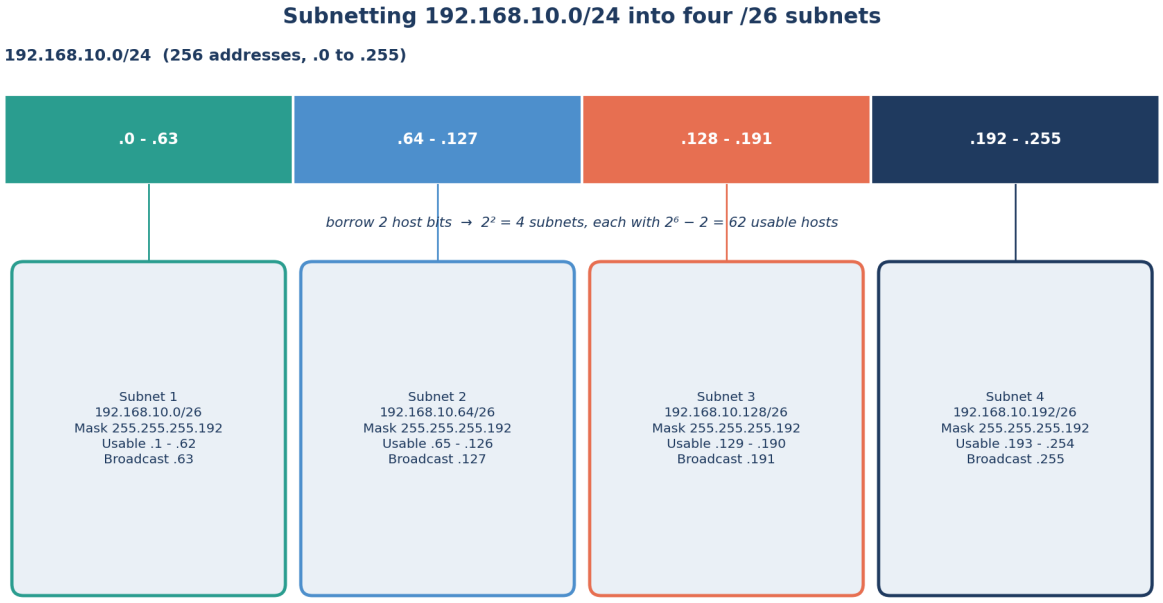


Figure 8. Splitting a /24 into four /26 subnets by borrowing 2 host bits.

Common masks to memorise

CIDR	Mask	Block size	Usable hosts	Typical use
/24	255.255.255.0	256	254	Standard LAN
/25	255.255.255.128	128	126	Half a /24
/26	255.255.255.192	64	62	Departments
/27	255.255.255.224	32	30	Small VLANs
/28	255.255.255.240	16	14	Server segments

CIDR	Mask	Block size	Usable hosts	Typical use
/30	255.255.255.252	4	2	Point-to-point router links

```
ipcalc 192.168.10.0/26                  # Linux (install the ipcalc package)

# Python: list the four /26 subnets of a /24
python3 - << 'EOF'
import ipaddress as ip
net = ip.ip_network("192.168.10.0/24")
for sub in net.subnets(prefixlen_diff=2):
    h = list(sub.hosts())
    print(sub, h[0], h[-1], sub.broadcast_address)
print(ip.ip_address("192.168.10.130") in ip.ip_network("192.168.10.128/26"))   # True
EOF
```

6. Private IP Addresses

RFC 1918 reserves three ranges that anyone may use inside their own network. They are **not routable on the public Internet**, so thousands of organisations can reuse the same addresses.

Range	CIDR	Addresses	Typical use
10.0.0.0 to 10.255.255.255	10.0.0.0/8	16.7 million	Enterprises, cloud VPCs
172.16.0.0 to 172.31.255.255	172.16.0.0/12	1 million	Medium networks; Docker default bridge uses 172.17.x.x
192.168.0.0 to 192.168.255.255	192.168.0.0/16	65,536	Home and small office

Other special-purpose IPv4 ranges

Range	Meaning
169.254.0.0/16	Link-local (APIPA). You see it when DHCP fails. Cloud providers also use 169.254.169.254 for the instance metadata service
100.64.0.0/10	Carrier-grade NAT (shared address space used by ISPs)
127.0.0.0/8	Loopback
192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24	Documentation and examples only

To reach the Internet, private hosts use **NAT** (Network Address Translation). The router rewrites the source address and port, and remembers the mapping so replies find their way back.

Private addresses and NAT: many hosts share one public IP

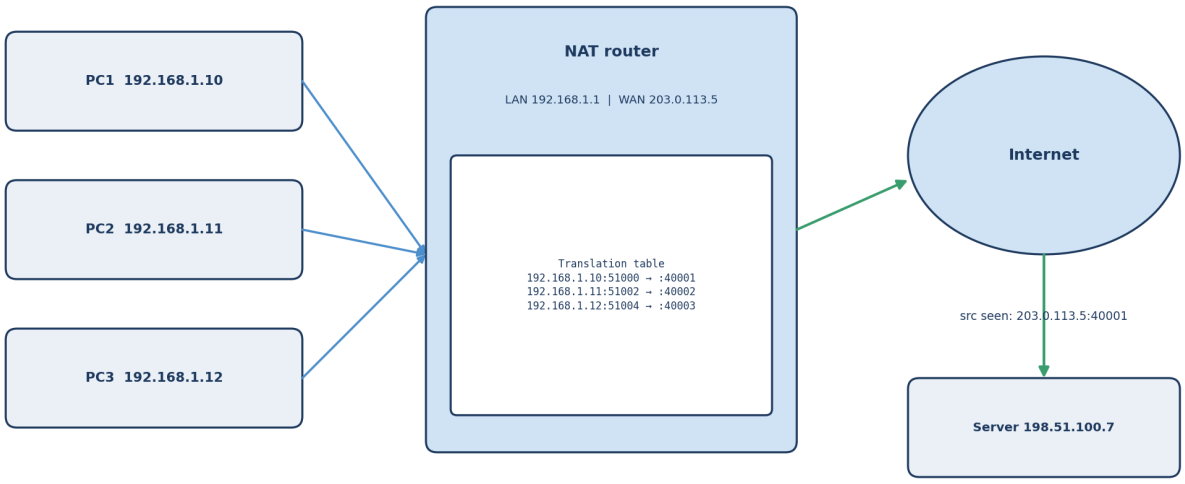


Figure 9. Three private hosts share the single public address 203.0.113.5. The router keeps a translation table.

Real-world references: private and link-local space

- **Cloud metadata abuse:** in the 2019 Capital One breach, a server-side request forgery (SSRF) weakness was used to reach the cloud metadata address 169.254.169.254 and obtain temporary credentials. Defences include blocking that address from application code and using the stricter metadata service version (for example AWS IMDSv2).
- **Lateral movement:** once an attacker is inside, they scan RFC 1918 ranges to find other hosts. Subnetting plus firewall rules between subnets limits how far they can go.
- **Address overlap:** two companies using 10.0.0.0/8 cannot connect directly (for example after a merger or over a VPN) without renumbering or NAT. Plan your addressing early.

7. Understanding IPv6 Addresses

IPv6 uses **128-bit addresses** written as eight groups of four hexadecimal digits, giving about 3.4×10^{38} addresses. There is **no broadcast** (multicast is used instead), NAT is not needed, and hosts can configure themselves automatically (**SLAAC**).

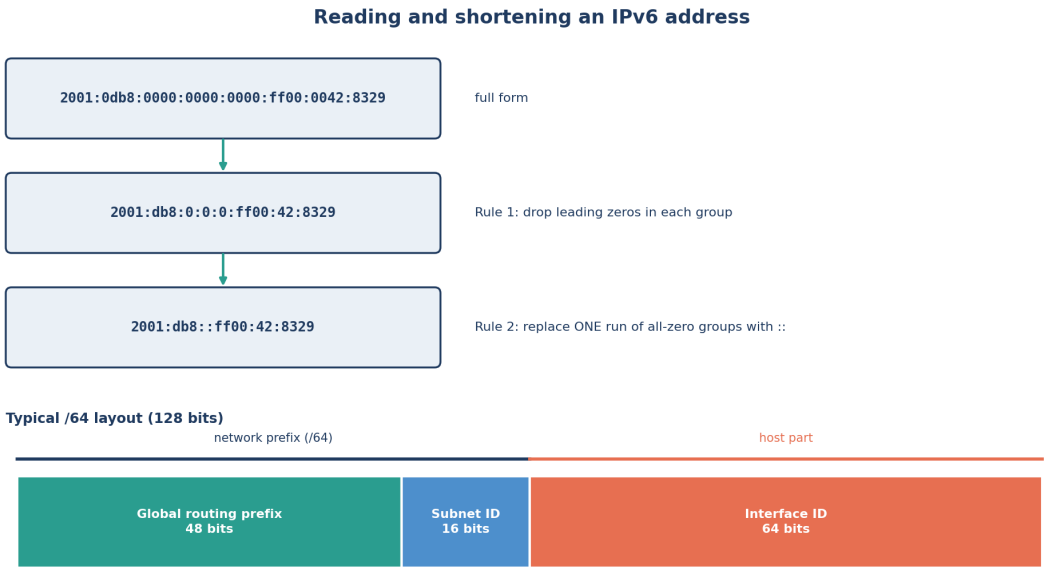


Figure 10. Two rules shorten any IPv6 address. A typical network uses a /64 prefix, leaving 64 bits for the interface.

Type	Prefix	Notes
Global unicast	2000::/3	Public, routable addresses
Link-local	fe80::/10	Created automatically on every interface; never routed beyond the link
Unique local (ULA)	fc00::/7 (use fd00::/8)	IPv6's equivalent of private addresses
Multicast	ff00::/8	ff02::1 = all nodes on link, ff02::2 = all routers
Loopback	::1	Same idea as 127.0.0.1
Unspecified	::	"No address yet"; used during autoconfiguration
Documentation	2001:db8::/32	Reserved for examples and labs

```
ip -6 addr show                  # Linux: show IPv6 addresses
ipconfig                         # Windows: shows IPv6 lines as well
ping -6 ::1                      # test the local IPv6 stack
ping -6 ff02::1%eth0             # discover all hosts on the link (multicast ping)
ip -6 neigh show                 # IPv6 neighbour table (replaces ARP)
ip -6 route show
traceroute -6 2001:4860:4860::8888
```

Real-world references: IPv6

Lesson: "we do not use IPv6" is not a defence. It is usually enabled anyway, so patch it, monitor it and manage it.

- **CVE-2024-38063 (Windows TCP/IP, IPv6):** disclosed 13 August 2024 with CVSS 9.8. The flaw is an integer underflow in how Windows handles IPv6 packets and needs no user interaction. IPv6 is enabled by default, so almost every modern Windows system was exposed. Public proof-of-concept code existed but did not achieve full code execution, and Microsoft had not observed exploitation at the time.
- Researchers pointed out that blocking IPv6 in the host firewall does not stop this bug, because the flaw triggers before the firewall processes the packet. The real fixes are the security update, or disabling IPv6 as a stop-gap where it is not needed (with possible side effects).
- **CVE-2020-16898 "Bad Neighbor"** (see Section 1) is another IPv6 and ICMPv6 parsing flaw in the same Windows stack.

8. Practical: Routing and Modes of Addressing

8.1 Modes of addressing

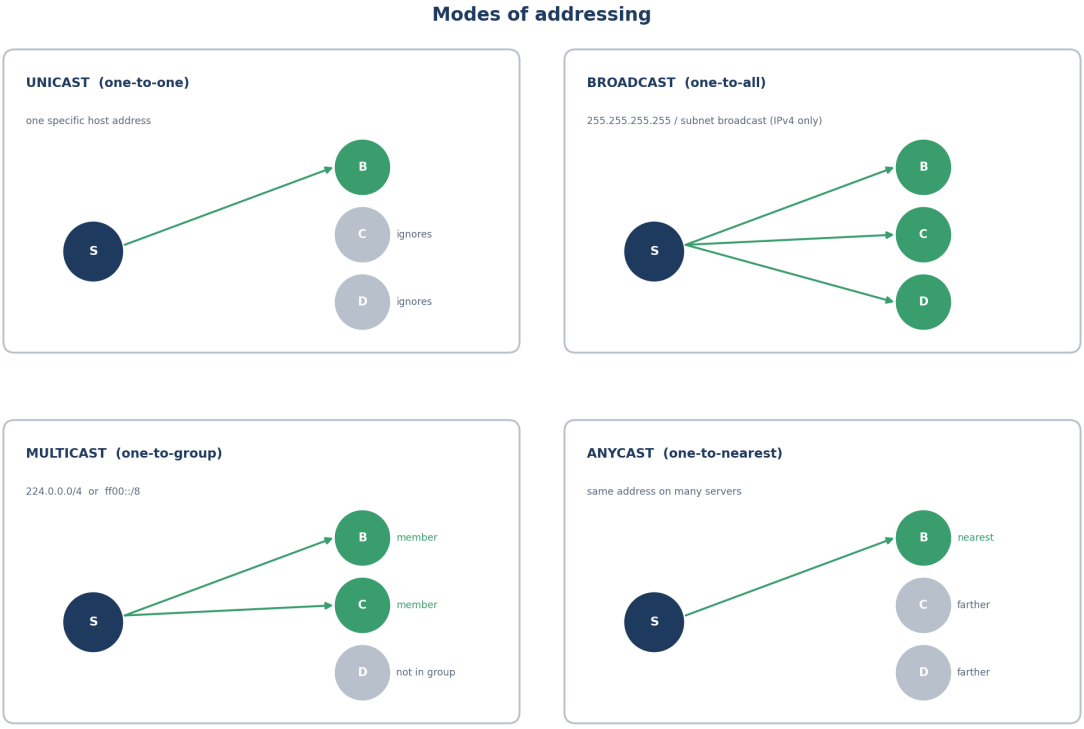


Figure 11. The four ways a packet can be addressed. Green nodes accept the packet.

Mode	Delivered to	IPv4 example	IPv6 example
Unicast	One specific host	192.168.1.20	2001:db8::20
Broadcast	Every host on the subnet	192.168.1.255 or 255.255.255.255	not available (multicast instead)
Multicast	Hosts that joined a group	224.0.0.0/4 (for example 224.0.0.251 mDNS)	ff00::/8 (for example ff02::1)
Anycast	The nearest of several hosts sharing one address	8.8.8.8 (Google DNS), root DNS servers, CDNs	same idea; built into IPv6

8.2 Hands-on routing lab (Linux network namespaces)

This lab builds two networks joined by a router, entirely inside one Linux machine, with no extra hardware. First nothing works, because the hosts do not know how to reach the other network. Then you add routes and watch ping and traceroute succeed.

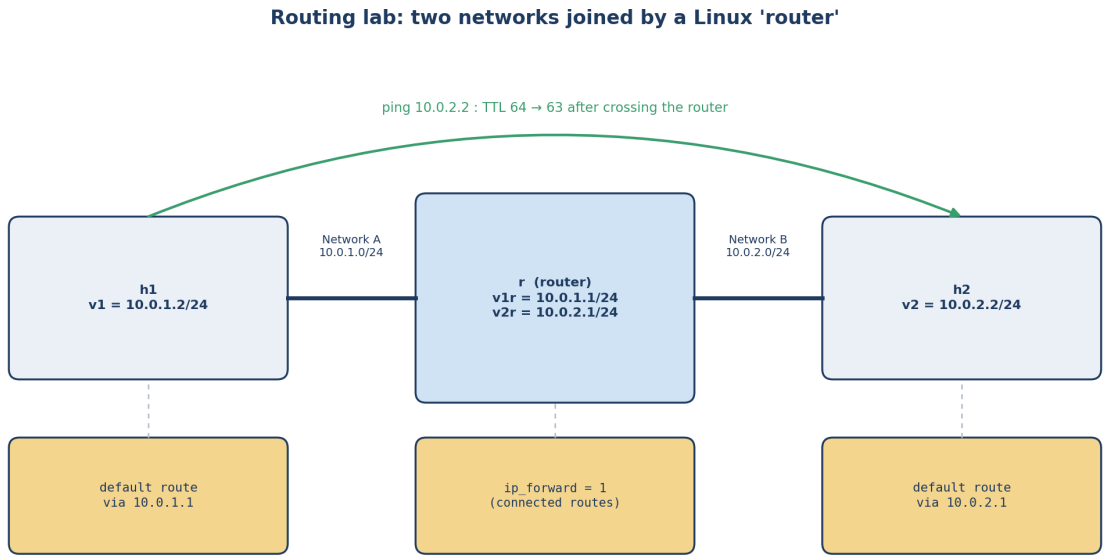


Figure 12. Lab topology: h1 and h2 are on different networks; the namespace r forwards packets between them.

```
# 1. Create three "machines" (network namespaces)
for n in h1 r h2; do sudo ip netns add $n; done

# 2. Cables (virtual ethernet pairs) and plug them in
sudo ip link add v1 type veth peer name v1r
sudo ip link add v2 type veth peer name v2r
sudo ip link set v1 netns h1;  sudo ip link set v1r netns r
sudo ip link set v2 netns h2;  sudo ip link set v2r netns r

# 3. Addresses
sudo ip netns exec h1 ip addr add 10.0.1.2/24 dev v1
sudo ip netns exec r  ip addr add 10.0.1.1/24 dev v1r
sudo ip netns exec r  ip addr add 10.0.2.1/24 dev v2r
sudo ip netns exec h2 ip addr add 10.0.2.2/24 dev v2

# 4. Bring every interface up
sudo ip netns exec h1 ip link set v1 up
sudo ip netns exec r  ip link set v1r up
sudo ip netns exec r  ip link set v2r up
sudo ip netns exec h2 ip link set v2 up

# 5. Test BEFORE routing: fails with "Network is unreachable"
sudo ip netns exec h1 ping -c1 10.0.2.2

# 6. Enable forwarding on the router; give each host a default route
sudo ip netns exec r  sysctl -w net.ipv4.ip_forward=1
sudo ip netns exec h1 ip route add default via 10.0.1.1
sudo ip netns exec h2 ip route add default via 10.0.2.1

# 7. Test AFTER: works, and traceroute shows the router as hop 1
sudo ip netns exec h1 ping -c2 10.0.2.2
sudo ip netns exec h1 traceroute -n 10.0.2.2

# 8. Watch ICMP on the router while you ping (open a second terminal)
sudo ip netns exec r tcpdump -ni v1r icmp

# 9. Clean up
for n in h1 r h2; do sudo ip netns del $n; done
```

What to observe

- The **routing table** picks the route with the **longest matching prefix**; the default route (0.0.0.0/0) matches everything but is the last resort. Inspect it with `ip route`.

- The TTL of replies drops by 1 at each router (64 → 63 in Figure 12).
- Try a **static route**: remove the default route and add `ip route add 10.0.2.0/24 via 10.0.1.1` on h1.
- Try IPv6: add `fd00:1::2/64` and `fd00:2::2/64` style addresses, enable `net.ipv6.conf.all.forwarding=1` on r, and repeat with `ping -6`.
- Windows equivalents: `route print`, `route add 10.0.2.0 mask 255.255.255.0 10.0.1.1`, `tracert`, `netstat -rn`.

9. IPv4 vs. IPv6

Feature	IPv4	IPv6
Address size	32-bit	128-bit
Notation	Dotted decimal (192.168.1.1)	Hexadecimal with colons (2001:db8::1)
Address space	about 4.3 billion	about 3.4×10^{38}
Header	20+ bytes, variable (options)	40 bytes fixed; optional extension headers
Fragmentation	Routers or sender	Sender only (PMTUD is mandatory)
Broadcast	Yes	No (multicast instead)
Address resolution	ARP	Neighbor Discovery (ICMPv6)
Configuration	Manual or DHCP	SLAAC, DHCPv6 or manual
NAT	Widely required	Not needed
Header checksum	Yes	Removed (handled by upper layers)
IPsec	Optional	Supported (originally mandatory, now optional)
Role of ICMP	Helpful	**Essential** (NDP, SLAAC, PMTUD depend on it)

Teaching point

- Because IPv6 depends on ICMPv6, blocking all ICMPv6 breaks IPv6. That is why bugs such as Bad Neighbor and CVE-2024-38063 matter: the attack surface lives in a protocol you cannot simply switch off. Filter selectively instead (for example, drop Router Advertisements from untrusted ports using RA Guard on switches).

10. Industry Reference Summary

Reference	Topic	What happened	Defender takeaway
CVE-2024-38063 (Aug 2024)	IPv6	Windows IPv6 packet handling integer underflow; zero-click remote code execution, CVSS 9.8	Patch fast; IPv6 is on by default; host firewall is not a fix
CVE-2020-16898 Bad Neighbor (Oct 2020)	ICMPv6	Router Advertisement with RDNSS option of even length leads to code execution, CVSS 8.8	Patch; filter rogue RAs (RA Guard); IDS signature on RA type 134 with option 25
CVE-2018-5391 FragmentSmack	Fragmentation	Tiny-fragment stream exhausts the reassembly queue; denial of service	Patch; tune fragment queue limits; rate-limit fragments
CVE-2021-24074, 24086, 24094	Fragmentation	Windows tcpip.sys reassembly flaws (potential RCE and DoS)	Complex parsing code creates bugs; patch promptly
PingPull (Gallium)	ICMP C2	Backdoor uses Echo Request and Reply packets for command and control	Inspect ICMP; alert on size and frequency anomalies
Pingback	ICMP C2	Windows malware parses commands from inbound ICMP packets by sequence number	Block unneeded inbound ICMP; monitor DLL side-loading
BPFDoor variants	ICMP relay	Linux backdoor variants use an ICMP relay and stateless callbacks	Hunt for raw-socket and BPF filters on servers
Capital One (2019)	Private / link-local	SSRF used to reach the 169.254.169.254 metadata service and obtain credentials	Block metadata from app code; use IMDSv2; least privilege

The items above are the ones verified while preparing this guide. New vulnerabilities appear constantly, so check the latest advisories (Section 13) before teaching or presenting.

11. Command Cheat Sheet

Task	Linux / macOS	Windows
Show IP addresses	`ip addr`	`ipconfig /all`
IPv6 addresses only	`ip -6 addr`	`ipconfig` (read the IPv6 lines)
Ping	`ping -c 4 host`	`ping -n 4 host`
Don't-fragment / MTU test	`ping -M do -s 1472 host`	`ping -f -l 1472 host`
Traceroute	`traceroute -n host`	`tracert -d host`
Path MTU per hop	`tracepath host`	use `ping -f -l` with decreasing sizes
Show routes	`ip route`	`route print`
Add a static route	`sudo ip route add 10.10.0.0/16 via 192.168.1.1`	`route add 10.10.0.0 mask 255.255.0.0 192.168.1.1`
Neighbour table (ARP / NDP)	`ip neigh`	`arp -a` and `netsh interface ipv6 show neighbors`
Capture ICMP	`sudo tcpdump -ni eth0 icmp`	Wireshark filter: `icmp`
Capture fragments	`sudo tcpdump -ni eth0 'ip[6:2] & 0x3fff != 0'`	Wireshark: `ip.flags.mf==1 || ip.frag_offset>0`
Subnet calculator	`ipcalc 192.168.10.0/26`	Python `ipaddress` module
Disable ICMP redirects	`sysctl -w net.ipv4.conf.all.accept_redirects=0`	`netsh interface ipv4 set global icmpredirects=disabled`

12. Practice Questions

1. Find the network ID, broadcast address and number of usable hosts for **172.16.45.200/20**.
2. How many usable hosts does a **/27** provide?
3. Divide **10.0.0.0/24** into subnets that each hold at least 50 hosts. Which mask do you use and how many subnets result?
4. Compress **2001:0db8:0000:0000:0000:0000:0000:0001**.
5. A 4000-byte packet crosses a 1500-byte MTU. The first fragment carries 1480 data bytes. What Fragment Offset value does the second fragment carry?
6. Is **172.32.1.1** a private address?
7. Which ICMP type does a router send when a packet's TTL reaches 0?
8. In a traceroute, hop 5 shows *** * *** but hops 6 to 9 reply normally. What does that most likely mean?
9. A host shows an address starting with **fe80::**. What kind of address is it?
10. Name two reasons why blocking all ICMP at the firewall is a bad idea.

Answers

1. Mask 255.255.240.0 (block size 16 in the third octet). 45 falls in block 32, so network **172.16.32.0**, broadcast **172.16.47.255**, **4094** usable hosts.
2. **30** ($2^5 - 2$).
3. Use **/26** (mask 255.255.255.192, 62 hosts each). You get **4 subnets**: .0, .64, .128, .192.
4. **2001:db8::1**
5. **185** (1480 divided by 8).
6. **No.** The private block is 172.16.0.0 to 172.31.255.255.
7. **Type 11, Time Exceeded.**
8. That router does not send (or filters) Time Exceeded messages, but still forwards traffic. The path itself is fine.
9. A **link-local** IPv6 address, created automatically and valid only on the local link.
10. Path MTU Discovery breaks (large transfers stall) and ICMPv6 is required for IPv6 to function. Also, troubleshooting becomes much harder.

13. References and Further Reading

Advisories and write-ups used for the real-world examples

- **CVE-2024-38063 overview (Censys)**
  https://censys.com/cve-2024-38063/
- **CVE-2024-38063 analysis (Blumira)**
  https://www.blumira.com/blog/cve-2024-38063-windows-tcp/ip-remote-code-execution-vulnerability
- **CVE-2020-16898 Bad Neighbor (McAfee / Trellix)**
  https://www.mcafee.com/blogs/other-blogs/mcafee-labs/cve-2020-16898-bad-neighbor/
- **Bad Neighbors Can Break Windows (LevelBlue SpiderLabs)**
  https://www.levelblue.com/blogs/spiderlabs-blog/bad-neighbors-can-break-windows-cve-2020-16898
- **IP fragment vulnerability CVE-2018-5391 (NCSC-FI)**
  https://kyberturvallisuuskeskus.fi/en/vulnerability-handling-ip-fragments
- **Windows network stack fragmentation bugs (McAfee Labs)**
  https://www.mcafee.com/blogs/other-blogs/mcafee-labs/researchers-follow-the-breadcrumbs-the-latest-vulnerabilities-in-windows-network-stack/
- **CVE-2000-0305 jolt2 (NVD)**
  https://nvd.nist.gov/vuln/detail/CVE-2000-0305
- **PingPull, Gallium ICMP backdoor (Unit 42)**
  https://unit42.paloaltonetworks.com/pingpull-gallium/
- **Pingback malware explained (Infosec Institute)**
  https://infosecinstitute.com/resources/malware-analysis/pingback-malware-how-it-works-and-how-to-prevent-it/
- **BPFDoor variants with ICMP relay (GBHackers summary of Rapid7 research)**
  https://gbhackers.com/bpfdoor-variants/amp/

Keep current

- **NVD vulnerability database**
  https://nvd.nist.gov
- **CISA Known Exploited Vulnerabilities catalog**
  https://www.cisa.gov/known-exploited-vulnerabilities-catalog
- **Microsoft Security Response Center**
  https://msrc.microsoft.com

Standards (search by number on rfc-editor.org)

- RFC 791 (IPv4), RFC 792 (ICMP), RFC 1918 (private addresses), RFC 4632 (CIDR)
- RFC 8200 (IPv6), RFC 4291 (IPv6 addressing), RFC 4443 (ICMPv6), RFC 4861 (Neighbor Discovery), RFC 8106 (RDNSS), RFC 6105 (RA Guard)